Classifying Imbalanced Bankruptcy Data: A Comparative Study of Threshold-Tuned Machine Learning Models

Visar Sokoli

May 2025

1 Introduction

The accurate prediction of corporate bankruptcy is an important task in finance, economics, and risk management. Early identification of financially unstable companies not only helps investors and shareholders mitigate potential losses but also allows regulators and financial institutions to implement preventive measures. In a world where all data types are becoming increasingly available, machine learning models have emerged as powerful tools to analyze complex financial patterns and predict firm failures with greater precision than traditional statistical techniques.

Bankruptcy prediction is inherently a challenging classification problem due to the highly imbalanced nature of real-world datasets, where bankrupt companies represent a small minority of the data available. Additionally, financial ratios and features often exhibit highly skewed distributions, multicollinearity, and complex non-linear relationships, further complicating model development. The data set that will be used for the majority of the paper is one from the UC Irvine Machine Learning Repository, and it provides thousands of observations of companies with a range of different company performance metrics and classifies them by whether the company went bankrupt or not.

This study explores the use of several supervised learning techniques, including Random Forest, Gradient Boost, and XGBoost, for the task of company bankruptcy classification. The paper will incorporate advanced techniques such as feature selection, imbalance handling with oversampling methods, transformations of skewed features, and model ensembling through stacking. The goal is not only to improve predictive performance—particularly F1-score, which measures the ability to predict positive cases of bankruptcy—but also to provide interpretable insights into the key financial indicators that signal high bankruptcy risk.

2 Data Description

The dataset contains over six-thousand instances of various companies reporting financial metrics and related data. The dataset comprises of ninety-two features and one target variable being an indicator that tells us if a company went bankrupt or not. Of the ninety-five features, all were numerical and typically were numbers between 0 and 1 because of the amount of financial ratios used in the dataset. One thing to note about the dataset and will come up multiple times in the paper, is that it is highly imbalanced. For example, the dataset contains exactly 6819 companies and only 220 of these companies were of the target class "Bankrupt". Later in the paper we will resolve this issue by applying oversampling techniques to expand the learning opportunities for our machine learning models.

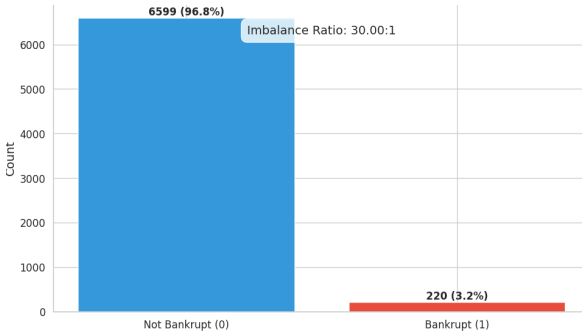
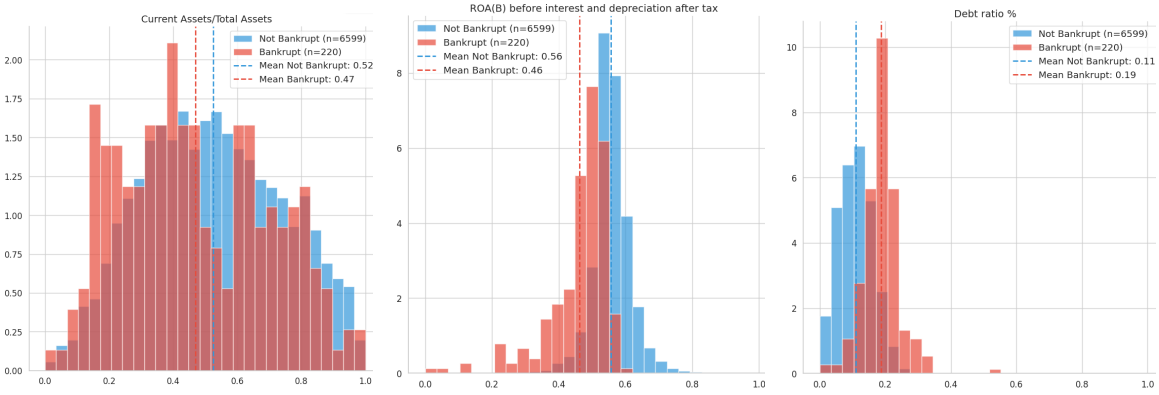


Figure 1: Class Distribution

Given the notable class imbalance in the dataset—with significantly fewer instances of bankrupt companies compared to non-bankrupt ones—it is essential to explore the underlying characteristics that distinguish these two classes. One effective approach is to visualize the distribution of critical financial metrics across bankrupt and non-bankrupt firms.

Among these, ROA(B) before interest and depreciation (Return on Assets before interest and depreciation) is a key profitability measure, indicating how efficiently a company is using its assets to generate earnings before certain expenses. Another important variable is the Current Assets to Total Assets ratio, which reflects a firm's liquidity and ability to meet short-term obligations. Finally, the Debt Ratio assesses a company's leverage, showing the proportion of assets financed through debt—a higher value generally suggests greater financial risk.

The following histograms illustrate how the distributions of these indicators differ between bankrupt and non-bankrupt companies, providing visual insight into the financial profiles associated with bankruptcy.



(a) Current Assets / Total Assets (b) ROA(B) (c) Debt Ratio (%)

Figure 2: Feature Histograms by Class

As seen in the histograms, depending on if a company went bankrupt or not, these features have different distributions and mean values which indicate that these features might be helpful in predicting bankruptcy in the future. For example, as we expected companies that are not bankrupt have a higher Current Asset to Total Asset Ratio and ROA(B) when compared to companies that went bankrupt. Whereas the Debt Ratio % has the opposite effect, we see that companies with higher debt ratios are more likely belong to the bankruptcy class. Which intuitively makes sense, because we expect companies that accumulate more debt are more likely to fail in the future for this reason.

In addition to understanding individual variable distributions, it is equally important to examine the relationships between features. One potential issue in financial datasets is multicollinearity, a phenomenon where two or more predictor variables are highly correlated with each other. Multicollinearity can be problematic for predictive modeling, particularly for algorithms that rely on feature weight interpretation (e.g., logistic regression, gradient boosting). It can inflate the variance of model coefficients, reduce model interpretability, and in some cases, degrade overall performance by introducing redundant information.

To assess multicollinearity in the dataset, a correlation heatmap was generated to visualize the strength of linear relationships between features. The figure below highlights several pairs of features with strong positive or negative correlations, indicating potential redundancy.

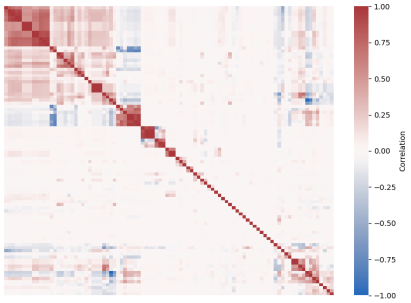


Figure 3: Clustered Feature Correlation Heatmap

As seen in the figure above, there are many pairs of features that are highly correlated with one another. Areas with a dark red shade mean the features are heavily positively correlated, meaning they are heavily related and could provide the same information when it comes to predicting bankruptcy. The same thing goes for the dark blue regions, even though they don't appear as much in the dataset, these are features that are heavily negatively correlated which each other and could end up providing the same information to the models when we eventually train them which could lead to a plethora of issues if not appropriately dealt with.

Lastly, another thing we can look at are the features that have the highest and lowest correlations with the target variable, meaning which features tend to increase and decrease the probability of a company going bankrupt respectively.

Table 1: Top Features Correlated with Bankruptcy

Feature	**Correlation**	**Feature**	**Correlation**
Debt Ratio %	0.25	ROA(B)	-0.27
Current Liability to Assets	0.19	Net Worth to Assets	-0.25
Current Liability to Current Assets	0.19	Retained Earnings to Total Assets	-0.22
Equity to Long-term Liability	0.15	Working Capital to Total Assets	-0.19
Current Liability to Equity	0.15	Net Income to Stockholder's Equity	-0.18

The correlation analysis between each feature and the target variable (bankruptcy) reveals that all absolute correlation values are below 0.25. This indicates that none of the individual features exhibit a strong linear relationship with the outcome of bankruptcy. In other words, no single financial metric in isolation is a strong predictor of bankruptcy in this dataset.

Such weak individual correlations suggest that bankruptcy is likely influenced by complex, nonlinear relationships between multiple variables rather than by any single factor. This further justifies the use of more advanced machine learning models, which are capable of capturing interactions and nonlinear patterns across multiple features.

3 Feature Engineering

In many real-world datasets, particularly in financial domains, features often exhibit significant skewness, where the majority of the values cluster near one end of the distribution while less and less values appear on the other end. Such skewed distributions can negatively impact the performance of many machine learning algorithms, especially those that assume normality or rely on distance-based metrics (e.g. boosting).

One effective strategy for addressing skewed data is logarithmic transformation. Applying the natural logarithm to highly skewed features helps compress the range, reduce the influence of extreme outliers, and bring the distribution closer to normal, allowing models to learn more effectively. This transformation can lead to better convergence, more stable training, and improved predictive performance.

3.1 Log Transformation of Skewed Features

To identify which features to transform, skewness was calculated for all continuous variables. **Skewness** is a measure of the asymmetry of a distribution. For a feature x, its skewness S is defined as:

$$S = \frac{\mathbb{E}[(x - \mu)^3]}{\sigma^3}$$

where:

- μ is the mean of the feature,
- σ is the standard deviation, and
- $\mathbb{E}$ denotes the expected value.

A skewness value greater than 1 typically indicates strong right skew. Features exceeding this threshold were selected for log transformation to reduce asymmetry and improve model learning. The transformation was applied as:

$$x_{\log} = \log(1 + x)$$

This version ensures numerical stability when features contain zeros or very small values. Applying the log transformation helped compress the range of values, reduce the impact of extreme outliers, and bring the feature distributions closer to normality, ultimately enhancing the effectiveness of the machine learning models. Out of 95 features, 61 of the features surpassed this threshold and the log transformation was applied to these features to tend their distributions closer to normality. To test if this step made feature distributions more normal we can take a look at how a feature distribution looked like before and after this transformation.

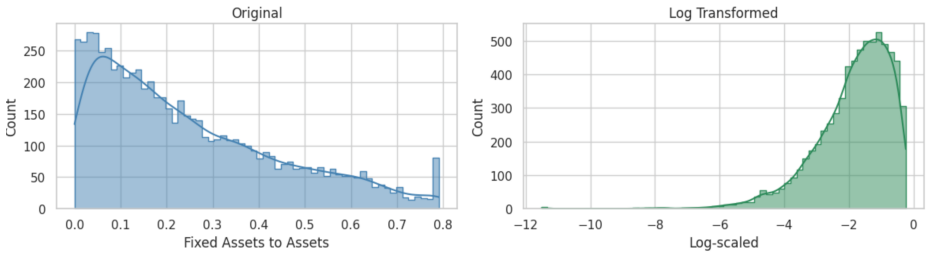


Figure 4: Histogram of 'Fixed Assets to Assets' (Original vs. Log Scaled)

The figure shows a right-skewed feature is transformed into a normal looking distribution after the log transformation which is what we were looking for in order to optimize model performance afterwards.

3.2 Removing Highly Correlated Features

While the log transformation addressed skewness in the data, another important issue to consider before modeling is multicollinearity. Multicollinearity occurs when two or more features in a dataset are highly linearly correlated, meaning one can be predicted from the others with a high degree of accuracy. This can confuse models by inflating the importance of redundant features and potentially degrading generalization performance.

To address this, the absolute pairwise Pearson correlation between all numerical features was calculated. When two features had a correlation coefficient greater than 0.90 in absolute value, one of them was removed to reduce redundancy. A threshold of 0.90 was chosen to ensure that only extremely similar features were dropped, minimizing the risk of losing valuable information.

By removing such highly correlated features, we aim to simplify the feature space, reduce overfitting, and improve the interpretability and efficiency of the models without compromising their predictive power. This process reduced the feature size of the dataset from the original 95 to 76 features.

4 Oversampling

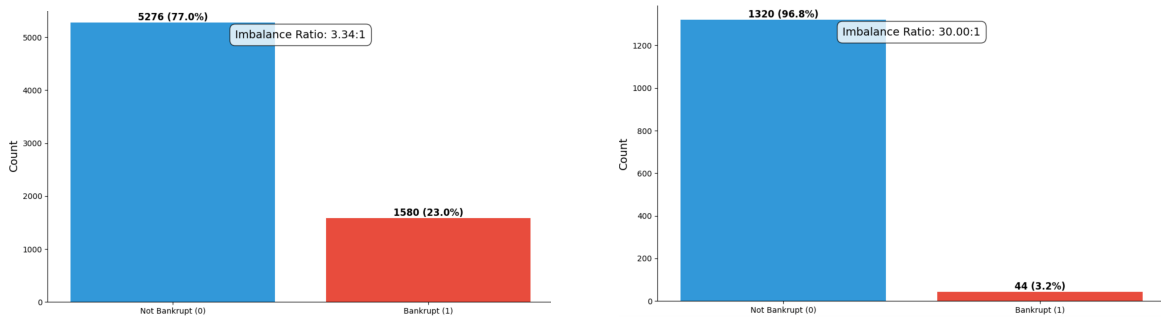
In highly imbalanced classification problems, such as bankruptcy prediction, the model may become biased toward predicting the majority class due to the overwhelming number of non-bankrupt samples. This can lead to poor performance in identifying the minority class, which is often the class of greatest interest. To mitigate this issue, oversampling techniques are commonly employed to rebalance the class distribution and give the model more opportunities to learn meaningful patterns from the minority class.

Before applying any oversampling method, the dataset was first split into training and test sets using an 80-20 stratified split to preserve the original class distribution. Additionally, all features were standardized using a *StandardScaler* to ensure that the feature values were on the same scale, which is important for distance-based oversampling methods such as SMOTE.

Several popular oversampling techniques were explored, including *SMOTE* (Synthetic Minority Oversampling Technique) and *ADASYN* (Adaptive Synthetic Sampling). A 5-fold cross validation was used to determine what oversampling technique and at what minority class ratio would yield the best results in terms of being able to predict companies that went bankrupt. Multiple oversampling techniques including ADASYN, SMOTE, SMOTE-Tomek were tested along with some minority class ratios of 20%, 30%, and 50% were also brought into the search to determine how much synthetic minority data should be added to the training set to increase model learning. Ultimately, the *SMOTE-Tomek* method was chosen due to its ability to both oversample the minority class and clean the decision boundary by removing Tomek links—instances that are ambiguously placed near the border between classes. This combination not only introduced synthetic minority samples to increase learning opportunities but also helped in reducing noise and potential class overlap, making the training distribution cleaner and more learnable.

Importantly, the SMOTE-Tomek technique was applied only to the training data to avoid data leakage and ensure the test set remained representative of real-world distributions. The application of this method significantly increased the representation of the minority class in the training set from approximately 3.2% to 23%, thereby providing the model with a more balanced and informative foundation for learning.

The figure shows that only the training data was changed by the oversampling technique used. This is because the training set is what builds the models and helps them learn relationships and



(a) Training set class distribution

(b) Test set class distribution

Figure 5: Class distributions in Training and Test Sets after Split

tendencies, while the test set was not changed and the minority class stuck at 3.2% because this is the data that will be used to determine how the different models will perform on unseen real data.

5 Model-Based Feature Selection

In this feature selection stage, the goal was to identify the most informative variables from the remaining 76 features after preprocessing. To achieve this, a model-based feature selection approach was used, where the importance of each feature is determined by how much it contributes to improving the accuracy of a predictive model. Features that demonstrated above-average importance were retained, while the rest were discarded to reduce noise and improve model efficiency. Note that because the threshold of the median was used to determine whether to keep or drop the feature in this technique-each model chose a feature set half the size of the original feature set.

This process was repeated using three different machine learning models—Random Forest, Gradient Boosting, and XGBoost—to ensure a fair and comprehensive assessment of feature relevance. Interestingly, all three models ended up selecting the exact same 38 features. While this might initially seem surprising, it is likely a reflection of the consistency in the underlying patterns of the data. Despite differences in how these models are built and trained, they all identified the same set of features as most predictive of bankruptcy outcomes.

This convergence is not only acceptable but actually reinforces confidence in the selection process. When different models agree on which features are most important, it suggests that those features carry strong, consistent signals that generalize well. Therefore, continuing the analysis with this shared subset of features provides a stable and reliable foundation for building final predictive models.

6 Model Development & Threshold Tuning

6.1 Introduction & Goals

Due to the significant class imbalance in the dataset, traditional accuracy is not a reliable measure of model performance, as it can be misleading when the majority class dominates. Instead, this study focuses on metrics that better capture the balance between identifying bankrupt companies (minority class) and avoiding false alarms. The primary metric used is the F1 score, which harmonizes precision and recall, providing a more meaningful evaluation for imbalanced classification. To further optimize model performance, each model will undergo threshold tuning specifically aimed at maximizing the F1 score. This approach ensures that the classification threshold is adjusted to achieve the best

possible balance between correctly detecting bankruptcies and minimizing misclassifications. Along with F1 score, we will be using three additional metrics to evaluate our models' performance:

1. F1 Score - Measures the balance between precision (the accuracy of positive predictions) and recall (the ability to find all positive instances). It is the harmonic mean of precision and recall, providing a single metric that captures both false positives and false negatives.

2. ROC AUC Score - Evaluates the model's ability to distinguish between classes by measuring the area under the Receiving Operating Characteristic (ROC) curve. It provides insight into a models performance across difference thresholds especially useful for imbalanced datasets.

3. Specificity - Also known as the true negative rate, it measures the proportion of actual negatives (non-bankrupt companies) that are correctly identified. High specificity means the model effectively avoids false alarms, which is crucial in contexts where wrongly classifying a healthy company as bankrupt can have serious consequences.

4. Precision at K (P@k) - Evaluates the precision among the top K predictions ranked by predicted probability. It shows how many of the K highest-risk predictions are actually positive cases. This metric is valuable when only a limited number of positive predictions can be acted upon, such as prioritizing companies for further financial review.

Using these four metrics ensures a comprehensive evaluation of a model's performance because together they capture different aspects of the model and its behavior.

6.2 Random Forests

Random Forest classification is a powerful and widely used ensemble learning technique that combines multiple decision trees to make more accurate and stable predictions. In a Random Forest model, each tree is built by randomly selecting subsets of the features and training data, which reduces overfitting and improves the model's ability to generalize to new data. The final prediction is made by aggregating the individual predictions from each tree, typically using majority voting for classification tasks. The algorithm excels in classification especially when the relationships between features and the target variable is non-linear. This method is also valuable for feature importance analysis, as it can assess the contribution of each feature to the model's predictive power.

In order to optimize an evaluation metric some type of parameter fine-tuning is done to machine learning models to optimize its performance on that metric. Usually this is done by using cross validation along with a grid search of different parameter values in order to search for the best combination of parameters that maximize the specific metric you decide you want to optimize. This method was first used for all of our models in hopes of increasing F1 score, but this method provided very minuscule improvement if at all. Due to this, parameter fine tuning was not done for any of the models, and instead F1 threshold tuning was performed on every model which significantly improved results.

F1 Threshold Tuning involves adjusting the decision threshold used to convert predicted probabilities into binary class labels in order to maximize the F1 score. Instead of using the default threshold of 0.5, the model evaluates multiple thresholds to find the one that best balances precision and recall for the specific dataset. This approach is especially useful for imbalanced datasets like this one, where the default threshold may lead to poor detection of the minority class (bankrupt companies). By tuning the threshold to maximize F1, we improve the model's ability to correctly identify bankrupt firms while minimizing false positives, resulting in a more effective and balanced prediction performance.

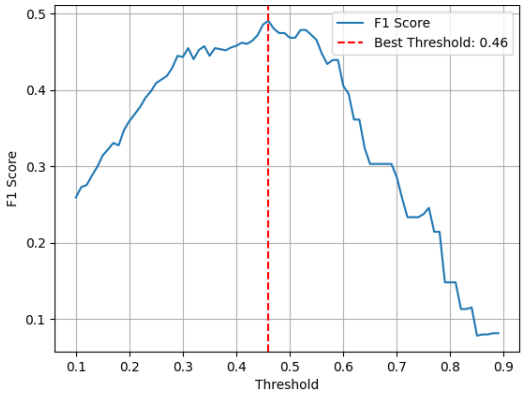
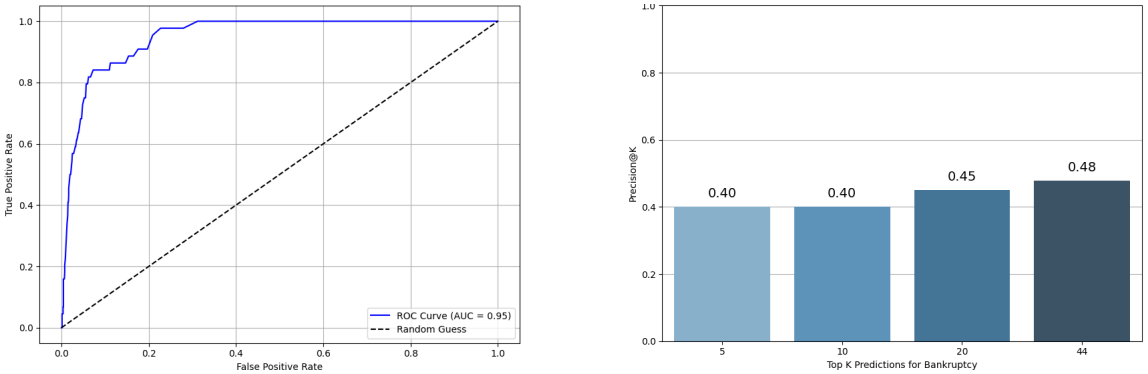


Figure 6: F1 Threshold Tuning for Random Forest

From the figure we see that the optimal threshold was not at the default 0.5, but instead at a value a little less than this. From this tuning we increase our F1 score by a value around 0.02. With this F1 threshold tuned Random Forest model we can finally test it on the test data and see how it performs based on the four metrics we want to evaluate.



(a) ROC Curve

(b) Precision@k Plot

Table 2: Random Forest Performance

Model	F1 Score	ROC AUC	Specificity
Random Forest	0.490	0.950	0.975

The random forest model provided a good baseline for what we can expect for the model we will test later on in the paper. An F1 score of 0.49 tells us that the model was not very effective in predicting bankrupt companies correctly and could use some improving in its ability to predict positive outcomes (bankruptcy). A ROC AUC score of 0.95 ensures that our model is able to perform across all thresholds, specifically it tells us that there is a 95% chance that a randomly chosen bankrupt company is higher ranked by our model than a randomly chosen non-bankrupt company.

The specificity score tells us that our model is able to correctly identify 97.5% of non-bankrupt companies. This sounds good, but is somewhat expected when 96.8% of all of our data are non-bankrupt firms. The more critical evaluation lies in its ability to identify bankruptcies — where performance was more modest.

This was further demonstrated by Precision@k scores hovering around $0.4 - 0.5$ at the top 5, 10, 20, and 44 predictions. These values suggest that although the model does surface some bankruptcies in its top predictions, there is room for improvement in prioritizing high-risk firms.

Overall, Random Forest laid a foundation for comparison but highlighted the need for more refined models to better capture the rare but important bankruptcy cases in the dataset. Despite its limitations in predictive performance compared to other models, one of Random Forest's strengths lies in its interpretability. The model naturally ranks features by importance, offering valuable insight into which financial indicators had the most influence on its decisions. The following chart highlights the top features based on their importance scores — offering a transparent look at what the model "focused on" when assessing bankruptcy risk.

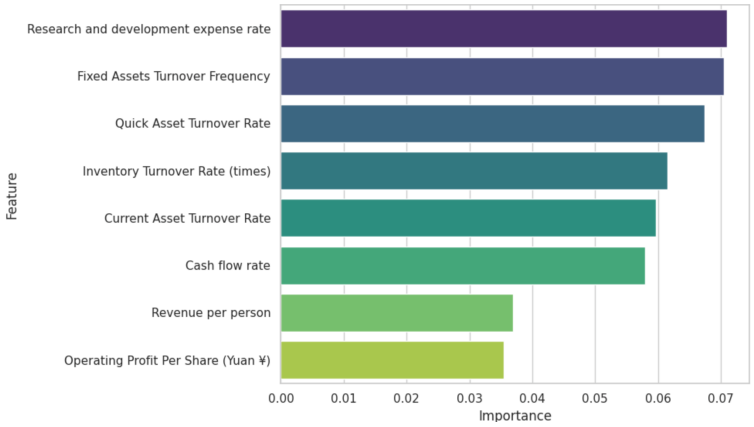


Figure 8: Top 8 Feature Importance - Random Forest

The top three features identified by the Random Forest model — **Research and Development Expense Rate**, **Fixed Assets Turnover Frequency**, and **Quick Asset Turnover Rate** — offer intuitive insights into a firms financial situation:

- **Research and Development Expense Rate** reflects how much a firm invests in innovation relative to its income. While R&D can signal long-term growth, excessive spending without near-term returns may slow down cash flow- potentially leading to financial instability in the long-run.
- **Fixed Assets Turnover Frequency** measures how efficiently a company utilizes its fixed assets to generate revenue. A declining turnover may suggest underutilized assets or deteriorating operational efficiency, both of which can signal weakening business fundamentals.
- **Quick Asset Turnover Rate** captures the speed at which a company converts its most liquid assets (excluding inventory) into revenue. Low performance here may reflect sluggish operations or liquidity issues, which are critical in evaluating a firm's short-term solvency.

Together, these features emphasize the model's reliance on *liquidity, efficiency, and financial discipline*—all central to identifying potential bankruptcy risk.

6.3 Gradient Boost

Gradient Boosting is an ensemble learning technique that builds a strong predictive model by sequentially combining multiple weak learners, typically decision trees. Each tree attempts to correct the mistakes of the previous ones by focusing more on the residual errors. This iterative optimization

process is guided by a gradient descent algorithm, which minimizes a chosen loss function. Gradient Boosting is especially effective in handling complex, non-linear relationships and can capture subtle patterns in the data. Its ability to fine-tune predictions through iterative refinement should make it a powerful choice for imbalanced classification tasks such as bankruptcy classification.

Again like in our previous model, fine-tuning this models parameters did not lead to any significant increase to our main evaluation metric - F1 score. Therefore, F1 threshold tuning was again utilized as the main way to improve F1 scores, leading to the models better understanding and predicting ability of the minority class (bankrupt companies).

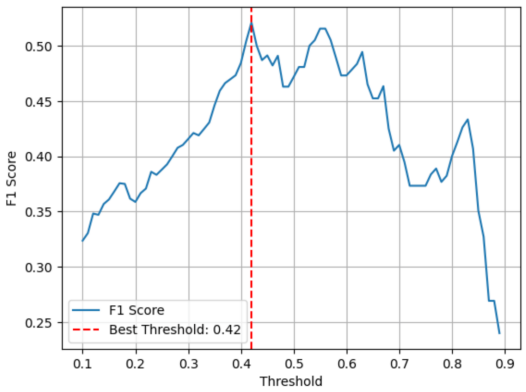
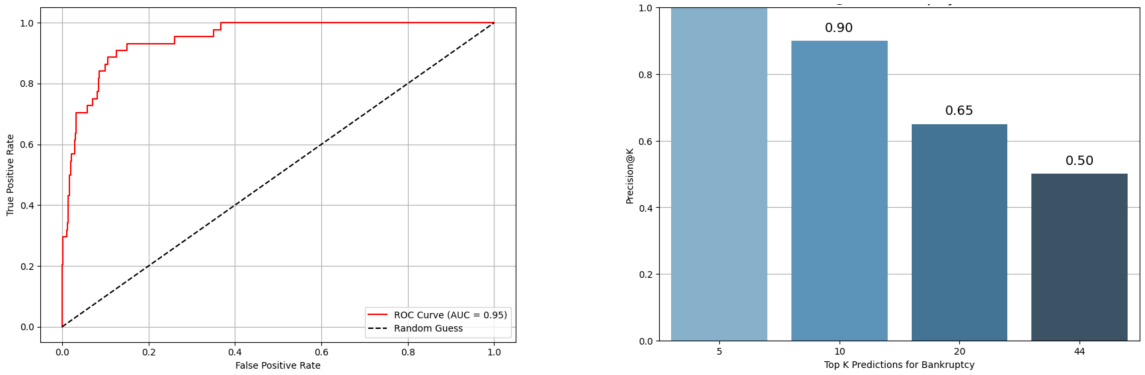


Figure 9: F1 Threshold Tuning for Gradient Boosting

From the figure we see that the optimal threshold was not at the default 0.5, but instead at a value a little less than this. From this tuning we increase our F1 score by a value around 0.04. With this F1 threshold tuned Gradient Boosting model we can finally test it on the test data and see how it performs based on the four metrics we want to evaluate.



(a) ROC Curve

(b) Precision@k Plot

Table 3: Gradient Boost Performance

Model	F1 Score	ROC AUC	Specificity
Gradient Boost	0.521	0.947	0.967

The gradient boosting model provided similar looking results when compared to the random forest model we used in the section before. This model was able to achieve a slightly higher F1

score, which is the main metric we are trying to optime, while performing slightly worse in ROC AUC score and specificity compared to the previous model. An F1 score of 0.521 tells us that the model is correct slightly more than half the time when it predicts a company will go bankrupt. It also tells us that the model is capturing a little more than half of the actual bankrupt companies in the dataset, which is a decent improvement to the last model.

However, where this model really succeeded was at its ability to correctly identify bankrupt companies at its top predictions. As seen in the Precision@k plot above, the gradient boosting model achieves drastically higher P@k scores than the previous model, making it a much more useful model in real-world situations because its able to consistently identify bankruptcies in its most confident predictions. From the plot we see that the models top 5 highest ranking predictions for bankruptcy all were bankrupt firms in the dataset. Additionally, we see that the model was able to correctly identify 9/10 bankrupt companies in its top 10 most confident predictions for bankruptcy. Basically, this means that when the model is very confident that a company is to go bankrupt it is rarely mistaken, making it a great model to utilize in the real-world situations.

Overall, at first glance gradient boosting performed at a similar level to the random forest model, but with a deeper look into the metrics it was able to produce much higher predicting power when it comes to correctly identifying bankrupt companies. Like the previous model gradient boosting also has the ability to naturally rank the features by importance, offering valuable insight into which financial indicators had the most influence on its decisions. The following chart highlights the top features based on their importance scores — offering an insight of what features the model is using the most to make its predictions.

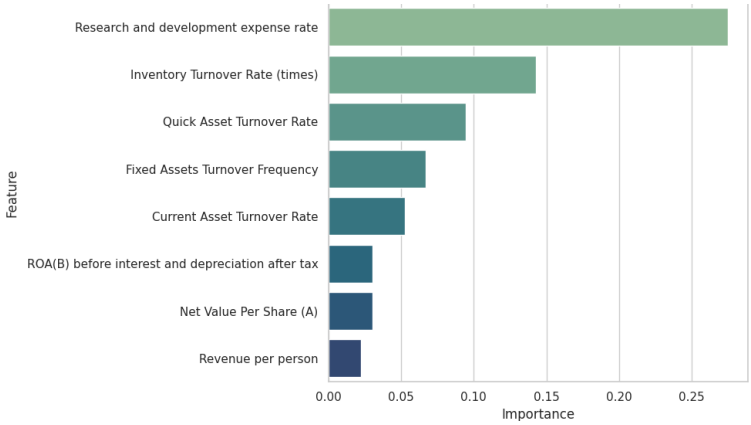


Figure 11: Top 8 Feature Importance - Gradient Boost

For the Gradient Boosting model, the two most influential features in predicting bankruptcy were the Research and Development Expense Rate and the Inventory Rate. Notably, the Research and Development Expense Rate stood out as the single most important predictor, contributing substantially more to the model's decision-making process than any other feature. Its high importance suggests that companies investing disproportionately in R&D relative to their earnings may exhibit financial behaviors that strongly correlate with future bankruptcy. The Inventory Rate followed as the second most critical feature, indicating that the management of inventory levels also plays a key role in assessing financial health.

For the Gradient Boosting model, the two most influential features in predicting bankruptcy were the **Research and Development Expense Rate** and the **Inventory Turnover Rate**. Among these, the Research and Development Expense Rate, like in the last model, stood out as the most dominant predictor. Its high importance score suggests that companies with unusually high or low spending on R&D relative to revenue may exhibit financial instability patterns that the model

strongly associates with bankruptcy risk. The Inventory Turnover Rate also played a significant role, indicating that how efficiently a company moves its inventory may be linked to its overall financial health.

6.4 XG Boost

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that builds multiple decision trees to improve performance. It sequentially adds trees, each focused on correcting the errors of the previous ones, by minimizing a specific loss function. XGBoost is an efficient machine learning model built for speed and performance, with added features like regularization to prevent overfitting. Its final predictions are made by combining the outputs of all trees, typically through weighted averages. Its ability to learn complex and non-linear relationships between features and the target variable makes it a great candidate in predicting bankruptcy.

Again like in our previous model, fine-tuning this models parameters did not lead to any significant increase to our main evaluation metric - F1 score. Therefore, F1 threshold tuning was again utilized as the main way to improve F1 scores, leading to the models better understanding and predicting ability of the minority class (bankrupt companies).

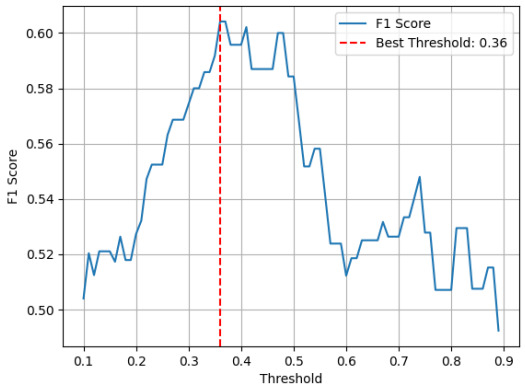
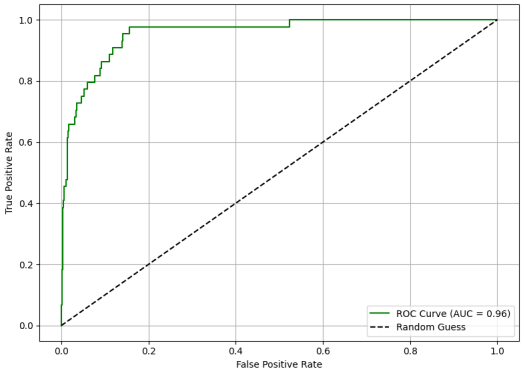
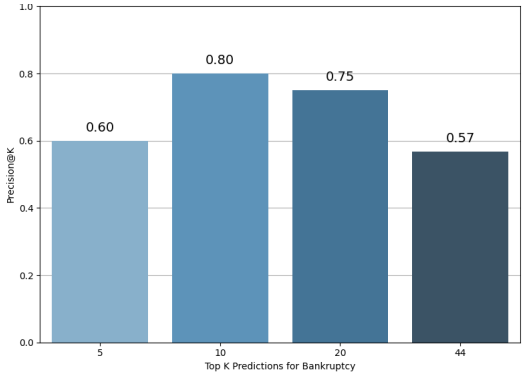


Figure 12: F1 Threshold Tuning for XG Boosting

From the figure we see that the optimal threshold was not at the default 0.5, but instead at a value less than this. From this tuning we increase our F1 score by a value around 0.02. With this F1 threshold tuned XG Boosting model we can finally test it on the test data and see how it performs based on the four metrics we want to evaluate.



(a) ROC Curve



(b) Precision@k Plot

Table 4: XG Boost Performance

Model	F1 Score	ROC AUC	Specificity
XG Boost	0.604	0.957	0.983

The XG boosting model provided the best results in almost every metric when compared to the previous models we used. This model was able to achieve a jump higher in F1 score, which is the main metric we are trying to optimize, while still performing slightly better in ROC AUC score and specificity compared to the previous models. An F1 score of 0.604 tells us that the model is correct more than half the time when it predicts a company will go bankrupt. It also tells us that the model is capturing more than half of the actual bankrupt companies in the dataset, which is a sizable improvement to the last models. So far, in terms of pure prediction power, the XG boost model is able to correctly identify the minority class (Bankrupt firms) better than any model previously mentioned.

In terms of its top prediction power (P@k), it isn't able to compete with the gradient boosting models' proficiency in correctly identifying bankrupt companies that it is the most confident in being bankrupt. However, it is able to more consistently identity bankrupt companies in its most powerful predictions on average. For example, in XG boostings 20 highest ranking companies in terms of bankruptcy risk, it correctly identifies 15/20 which beats out gradient boostings 13/20 metric. In terms, of its real-life use it would not be as effective because it is not able to achieve 100% precision in correctly identifying bankrupt companies in its top 5 most confident predictions as gradient boost was able to. However, this XG boost model is the best performing and most robust model in terms of correctly predicting both bankrupt and non-bankrupt companies.

XG Boosting model also have the capabilities to display feature importance, which show which features had the most effect in predicting the target variable in our case- bankruptcy. However, this model can also show exactly what kind of effect the different samples in your dataset had on making predictions on the target variable. A SHAP plot (SHapley Additive exPlanations) visualizes how much each feature contributes to a model's prediction, either pushing it toward or away from a particular class. It provides local interpretability, showing the impact of each feature for individual predictions, as well as global importance when aggregated across the dataset.

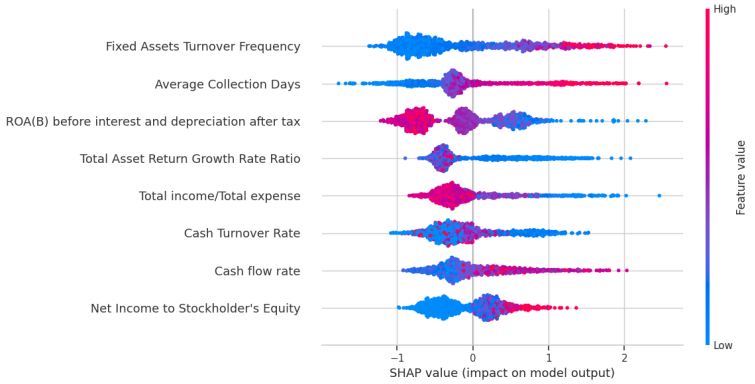


Figure 14: Top 8 Feature SHAP Plot - XG Boost

From the plot we not only see which features were the most important in terms of making predictions for bankruptcy, but also the exact effect each individual sample point had on the models prediction making ability. From the plot we see that **Fixed Assets Turnover Frequency** was the feature with the most importance, and we see how companies with relatively low, medium, or high values in this category had on the models prediction for bankruptcy. Specifically for this first

feature, we see that companies with relatively high values were treated as likely to go bankrupt with a high effect, while companies with low value in this feature were more likely to be predicted as non-bankrupt but on a lower effect.

6.5 Stacking

Stacking, or stacked generalization, is an ensemble learning method that combines the predictive strengths of multiple base models to improve overall performance. Unlike techniques such as bagging or boosting that combine similar models, stacking blends predictions from diverse algorithms. In this project, the stacking model integrates three strong learners: Random Forest, Gradient Boosting, and XGBoost. The models generate predictions that are then passed to a meta-learner ,which in this case is a Logistic Regression classifier. The meta-learner is trained on the out-of-fold predictions of the base models, learning to optimally combine their outputs. This layered approach allows the stacking classifier to capture patterns that individual models might miss, making it particularly valuable for imbalanced classification problems like bankruptcy prediction. Like our previous three models, F1 threshold tuning was also utilized for the Stacking Classifier to obtain an optimal F1 measure from the threshold chosen for the classifier.

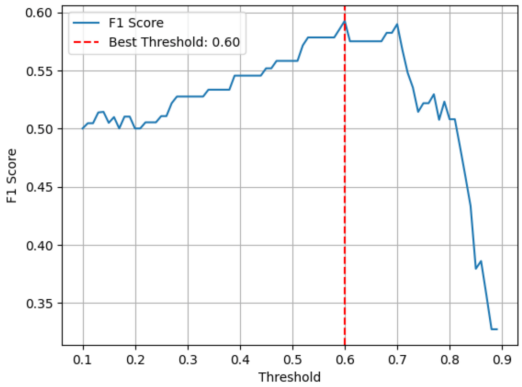
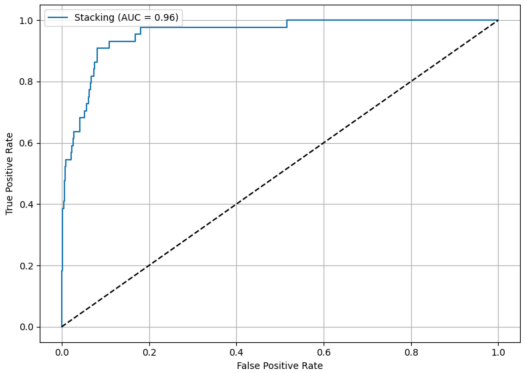
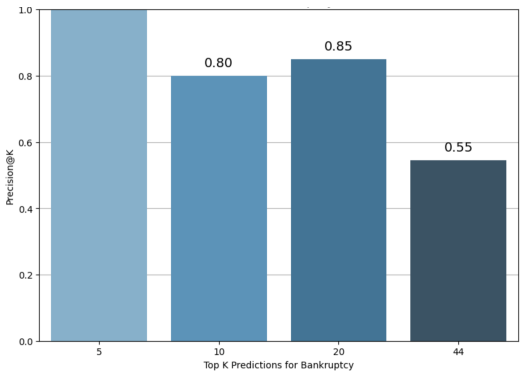


Figure 15: F1 Threshold Tuning for Stacking

From the figure we see that the optimal threshold was not at the default 0.5, but instead at a value greater than this. From this tuning we increase our F1 score by a value around 0.04. With this F1 threshold tuned Stacking model we can finally test it on the test data and see how it performs based on the four metrics we want to evaluate.



(a) ROC Curve



(b) Precision@k Plot

Table 5: Stacking Performance

Model	F1 Score	ROC AUC	Specificity
Stacking	0.593	0.957	0.990

The Stacking model provided the best results in almost every metric when compared to the previous models we used. This is expected because the stacking model uses the best predictions from the models used before and makes its own predictions by learning off of what works and doesn't based on the meta-learner. The model achieved the highest ROC AUC score and specificity out of all models used in the paper. However, it performs slightly worse than the XG boosting model in terms of F1 score, but makes up for this discrepancy in its Precision@k scores that beat out most models tested before.

The stacking classifier demonstrated the strongest performance on the Precision at K (P@k) metric out of all models utizied. It achieved scores of 1.00, 0.80, 0.85, and 0.55 at $k = 5, 10, 20$, and 44, respectively. These results indicate that the model is highly effective at ranking the most likely bankruptcy cases at the top of its prediction list. A perfect precision of 1.00 at $k = 5$ means that all of the top five predictions made by the model were actual bankruptcy cases. Even at $k = 10$ and $k = 20$, the model maintains high precision, highlighting its strength in identifying high-risk firms early.

This is particularly valuable in real-world applications, where financial investigators or auditors may only have the capacity to review a limited number of cases. The stacking model's ability to prioritize true bankruptcies near the top of its prediction ranking suggests that it can be a powerful decision-support tool in practice, helping stakeholders focus their attention where it's most urgently needed.

7 Conclusion

The primary motivation for this project was to develop a robust predictive framework for identifying companies at risk of bankruptcy, a critical task given the significant financial and operational consequences associated with such events. Due to the highly imbalanced nature of the dataset—with only a small fraction of bankrupt firms—the project emphasized meaningful metric selection and model threshold tuning to ensure optimal evaluation.

We explored multiple supervised machine learning models including Random Forest, Gradient Boosting, XGBoost, and a final Stacking Classifier. These models were evaluated not just by standard metrics like F1-score and ROC AUC, but also by specificity and Precision at K (P@k), which provided a more practical lens on real-world applicability. F1 threshold tuning was applied across all models to better balance precision and recall, addressing the class imbalance directly.

We also employed model-agnostic feature selection to reduce noise and improve interpretability along with simplicity, which revealed consistent and domain-relevant predictors of bankruptcy. Furthermore, feature importance analysis gave insight into financial ratios most indicative of risk, helping bridge the gap between model output and financial understanding.

The results suggest that more sophisticated ensemble techniques—particularly stacking—demonstrate improved capability in prioritizing high-risk cases, especially when measured through real-world utility metrics like P@k.

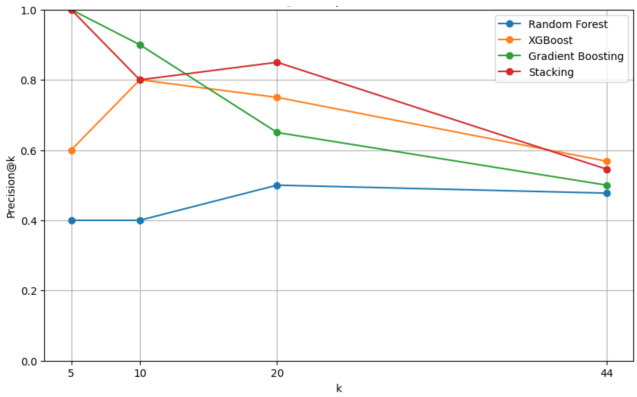


Figure 17: Precision at K Scores Across All Models

Table 6: Final Evaluation Metrics for All Models

Model	**F1 Score**	**ROC AUC**	**Specificity**	**P@44**
Random Forest	0.490	0.95	0.975	0.48
Gradient Boosting	0.521	0.947	0.967	0.50
XGBoost	**0.604**	**0.957**	0.983	**0.57**
Stacking CLassifier	0.593	**0.957**	**0.990**	0.55

The results demonstrate that while Random Forest and Gradient Boosting offered solid baseline performance, both XGBoost and the Stacking Classifier consistently outperformed the others across all major metrics. Notably, XGBoost achieved the highest F1-score, ROC AUC, and precision-at-k at various levels, indicating its strong potential for real-world deployment where identifying rare bankruptcy cases is critical.

F1 scores for the minority class consistently remained in the 0.5–0.6 range across all models. This aligns with prior studies using the same dataset, suggesting an underlying limitation in the feature set's predictive power rather than a shortcoming in modeling technique. Future work could explore additional data sources or domain-specific feature engineering to address this challenge. Overall, this analysis highlights the importance of using advanced ensemble methods and evaluation strategies when working with imbalanced classification problems in finance.

Appendix

Dataset

The dataset used in this analysis is the **Taiwanese Bankruptcy Prediction Dataset**, available at the UC Irvine online repository: `https://archive.ics.uci.edu/dataset/572/taiwanese+bankruptcy+prediction`

Implementation

All coding and plots were performed using **Python**. The following key libraries and functions were used frequently throughout the analysis:

- `pandas` – For data manipulation and preprocessing.
- `numpy` – For numerical operations.
- `matplotlib` and `seaborn` – For data visualization and plotting.
- `sklearn` – For model training, evaluation, preprocessing tools, feature selection, and cross validation.
- `xgboost` – For implementing and optimizing the XGBoost classification model.
- `shap` – For feature importance visualization and analysis.
- `tenserflow & keras` – For implementing and optimizing customized neural network.

Model Parameters

The parameters for the classification models used in this study are displayed below and *random_state=162* was used for all training.

- **Random Forest:**
  - `n_estimators` = 100
- **Gradient Boost:**
- **XGBoost:**
  - `use_label_encoders` = *False*
  - `eval_metric` = *'logloss'*
- **Stacking Classifier**
  - `final_estimator` = *LogisticRegression(C=0.1, penalty='l2', max_iter=2000)*